

Gradient Checkpointing

- **Strategy:** Store fewer intermediates, recompute during backward
- **Trade-off:** $\sqrt{n}$ memory reduction for $\sim 33\%$ computation increase
- **Example:** Store every k -th layer, recompute the rest
- **Implementation:** Framework manages storage/recomputation automatically
- **Result:** Enables training of much larger models



Redundant Computation Elimination

Identifying and Removing Unnecessary Work

Common Redundancy Patterns:

1. Duplicate Subexpressions:

```
# Redundant
loss1 = mse(predictions, targets)
loss2 = mse(predictions, targets) # Same computation

# Optimized
shared_loss = mse(predictions, targets)
loss1 = shared_loss
loss2 = shared_loss
```

2. Residual Connection Optimization:

```
# In ResNet blocks, common subexpressions appear
def residual_block(x):
    conv1_out = conv2d(x, weights1)
    conv2_out = conv2d(conv1_out, weights2)
    return x + conv2_out # x is reused
```

3. Constant Folding:

```
# Constants computed at compile time
scaling_factor = 1.0 / 255.0 # Computed once, not per
inference
normalized = input * scaling_factor
```

Identifying and Removing Unnecessary Work



Advanced Graph Transformations

Sophisticated Optimization Techniques

1. Loop Unrolling for RNNs:

```
# Original: Dynamic loop
for t in range(sequence_length):
    hidden[t] = rnn_cell(input[t], hidden[t-1])

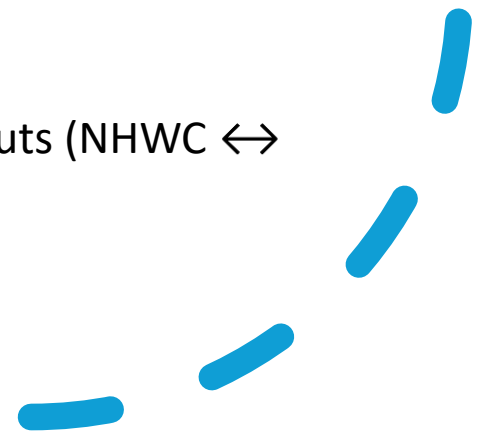
# Optimized: Unrolled for fixed sequence lengths
hidden[0] = rnn_cell(input[0], initial_hidden)
hidden[1] = rnn_cell(input[1], hidden[0])
hidden[2] = rnn_cell(input[2], hidden[1])
# ... enables further optimizations
```

2. Algebraic Simplifications:

```
# Matrix multiplication reordering
#  $A \times (B \times C)$  vs  $(A \times B) \times C$ 
# Choose order based on dimensions to minimize operations
```

3. Memory Layout Transformations:

Automatic conversion between different tensor layouts (NHWC ↔ NCHW) based on hardware preferences



Compiler-based Graph Optimization

Automated Transformation Systems

Pattern Recognition and Rewrite Rules:

```
# Compiler identifies patterns and applies transformations
pattern = "Conv2D → BatchNorm → ReLU"
replacement = "FusedConvBnRelu"

# Systematic application across entire graph
for subgraph in computation_graph:
    if matches_pattern(subgraph, pattern):
        replace_with(subgraph, replacement)
```

Multi-stage Optimization:

1. **High-level transformations:** Operation fusion, redundancy elimination
2. **Hardware-specific optimizations:** Layout transformations, kernel selection
3. **Low-level optimizations:** Instruction scheduling, register allocation

Optimization Verification:

Ensure mathematical equivalence through formal verification or numerical testing.

